

LicenseWise — System Behavior Documentation

Knowledge Base Systems · Testing Branch · Spring 2026

1. Overview

LicenseWise is a knowledge-based expert system for software license recommendation and compatibility analysis. It implements two inference strategies: forward chaining for recommendation and backward chaining for compatibility checking. The system encodes license properties from the SPDX standard into a rule base and applies them against user-supplied facts to produce traceable recommendations.

This document describes the current system behavior for the testing branch, including the inference algorithms, working memory structure, rule organization, and explanation facility.

2. Architecture

The system is organized into four layers:

Layer	Module	Responsibility
Interface	interface/cli.py	Interactive CLI; collects user facts via prompted questions
Inference	inference/engine.py	forward_chain() and backward_chain() implementations
Explanation	inference/explanation.py	format_trace(), generate_final_report(), generate_summary()
Knowledge	knowledge/rules.py + licenses.json	57 IF-THEN rules; 9 SPDX license definitions
Entry point	main.py	Dispatches to --recommend or --analyze mode
Tests	tests/test_scenarios.py	Unit tests for sample use-case scenarios

3. Knowledge Base

3.1 License Definitions (licenses.json)

Each license is stored as a JSON object with the following schema:

```
{ "id": "MIT", "spdx_id": "MIT", "conditions": { "disclose_source":
false,
  "same_license": false, "net_copyleft": false },
  "permissions": { "commercial_use": true }, "limitations":
{ "patent_use": false } }
```

Nine licenses are currently modeled: MIT, Apache-2.0, GPL-3.0, AGPL-3.0, LGPL-2.1, LGPL-3.0, MPL-2.0, BSD-2-Clause, CC-BY-NC-4.0, Unlicense, CC0-1.0.

3.2 Rule Base (rules.py)

Rules are built by `build_rules()` and exposed as the module-level constant `RULES`. Each rule is a Python dict with the following keys:

Key	Type	Description
id	str	Unique rule identifier, e.g. A01
name	str	Human-readable snake_case name
condition	lambda (facts, licenses_data) → bool	Predicate evaluated against working facts
action	lambda (wm) → None	Mutates working memory when condition is true
explanation	str	Plain-English reason shown in the trace
action_type	str	One of: RECOMMEND, ELIMINATE, WARN

Rules are grouped into five categories:

Group	Covers	Rules
A	Permissive licenses (MIT, Apache-2.0, BSD-2-Clause, Unlicense, CC0)	A01–A13
B	Strong copyleft (GPL-3.0, AGPL-3.0)	B01–B07
C	Weak / file-level copyleft (LGPL-2.1/3.0, MPL-2.0)	C01–C04
D	Content licenses (CC-BY-NC-4.0)	D01–D02
E	General elimination (private modifications)	E01

Three helper predicates are defined at module level to keep conditions DRY:

- `_saas(facts)` — true if `saas` or `network_saas` is set
- `_copyleft(facts)` — true if `want_copyleft` or `wants_derivatives_open` is set
- `_patent(facts)` — true if `need_patent_protection` or `patent_protection_needed` is set
- `_private_mods(facts)` — true if `closed_source`, `wants_relicense`, or `wants_to_keep_modifications_private` is set

4. User Facts

The facts dict is keyed by fact name. Values are bool, str, or None (unanswered). The CLI collects facts and builds the dict before invoking either inference function.

Fact	Type	Used by
<code>closed_source</code>	bool	Forward + Backward chain
<code>saas / network_saas</code>	bool	Forward + Backward chain
<code>commercial_use</code>	bool	Forward + Backward chain
<code>need_patent_protection / patent_protection_needed</code>	bool	Forward + Backward chain
<code>want_copyleft / wants_derivatives_open</code>	bool	Forward chain
<code>want_weak_copyleft</code>	bool	Forward chain
<code>want_file_copyleft</code>	bool	Forward chain
<code>wants_relicense / wants_to_keep_modifications _private</code>	bool	Forward + Backward chain
<code>project_type</code>	str (software/library/content)	Forward chain
<code>want_public_domain</code>	bool	Forward chain
<code>want_simple_permissive</code>	bool	Forward chain
<code>academic_project</code>	bool	Forward chain
<code>mixed_open_proprietary</code>	bool	Forward chain
<code>linking_type</code>	str (dynamic/static)	Forward chain
<code>modify_library</code>	bool	Forward chain
<code>concerned_about_legal_recognition</code>	bool	Forward chain

5. Forward Chaining — Recommendation Mode

5.1 Algorithm

`forward_chain(facts, rules, licenses_data, trace)` implements a single-pass, data-driven forward chaining strategy. It does not iterate to a fixed point — each rule is evaluated exactly once per call. This is appropriate because no rule produces a new fact that would alter another rule's condition; rules only modify working memory (sets and lists).

Step-by-step execution:

1. 1. Initialize working memory: recommended = set(), eliminated = set(), warnings = []
2. 2. For each rule in RULES:
 - a. Evaluate rule['condition'](facts, licenses_data)
 - b. If true: append a trace step dict; fire rule['action'](wm)
 - c. If false or exception: skip (continue)
3. 3. Finalize: recommended -= eliminated
4. 4. Return working memory dict

5.2 Working Memory Structure

```
wm = {
    "recommended": set(),      # licenses that satisfy user goals
    "eliminated": set(),      # licenses incompatible with user goals
    "warnings": list(),       # caution strings appended by WARN rules
}
```

The trace list is passed in by the caller and populated in place. Each appended entry has the shape:

```
{ step: int, rule_id: str, rule_name: str, matched_facts: dict,
  explanation: str, action: str }
```

5.3 Conflict Resolution

When a license appears in both recommended and eliminated (e.g. MIT added by A01 for closed_source, but later considered by another rule), the post-loop statement recommended -= eliminated removes it. This implements a simple ELIMINATE-overrides-RECOMMEND priority without requiring rule ordering.

5.4 Exception Handling

Rule conditions are wrapped in try/except. A rule that raises any exception (e.g. due to an unexpected fact type) is silently skipped. This keeps the engine robust against incomplete facts dicts.

6. Backward Chaining — Compatibility Analysis Mode

6.1 Algorithm

backward_chain(license_id, facts, licenses_data) checks a specific license against user facts. Unlike forward chaining, it starts from a goal (a named license) and verifies whether the user's facts satisfy all of that license's conditions.

Step-by-step execution:

3. 1. Look up license_info in licenses_data by id or spdx_id
4. 2. If not found, return compatible=False with an error explanation
5. 3. Check each condition/permission in sequence:
 - a. disclose_source AND closed_source → violation: source_disclosure_required
 - b. same_license AND wants_relicense → violation: same_license_required
 - c. net_copyleft AND saas AND closed_source → violation: network_copyleft_conflict
 - d. NOT commercial_use AND commercial_use fact → violation: commercial_use_prohibited
 - e. need_patent_protection AND NOT patent_use → warning (not a violation)
6. 4. Return: compatible=(violations==[]), violations list, explanation, how text, license_info

6.2 Return Schema

Key	Type	Description
compatible	bool	True if no violations found
violations	list[str]	Machine-readable violation codes
explanation	str	Human-readable conflict descriptions, newline-joined
how	str	Step-by-step reasoning text
license_info	dict None	The license record from the knowledge base

7. Explanation Facility

The explanation facility in inference/explanation.py provides three functions:

Function	Input	Output
explain_question(fact_name)	str	Why a CLI question is asked; 16 facts covered
format_trace(trace)	list[dict]	Formatted step-by-step reasoning for the terminal
generate_final_report(wm, facts, trace)	wm + facts + trace	Full report: recommended, eliminated, warnings, confidence, trace
generate_summary(wm, facts, trace)	wm + facts + trace	Concise summary grouped by action type

7.1 Confidence Heuristic

generate_final_report() computes a simple confidence level:

- HIGH — 8 or more facts provided AND at least one license recommended

- MEDIUM — 4 or more facts provided
- LOW — fewer than 4 facts provided

8. Validation Against Project Documentation

The following table cross-references key documentation claims in the README against the actual code:

README Claim	Status	Note
57 rules total	Verify	Count <code>build_rules()</code> output at runtime; current code shows ~30 rules; README may be aspirational
Hybrid forward/backward chaining	Confirmed	<code>forward_chain()</code> and <code>backward_chain()</code> are separate functions
9 licenses in scope	Confirmed	<code>licenses.json</code> lists MIT, Apache-2.0, GPL-3.0, AGPL-3.0, LGPL-2.1/3.0, MPL-2.0, BSD-2-Clause, CC-BY-NC-4.0, Unlicense, CC0
SPDX integration	Confirmed	Lookup uses both <code>id</code> and <code>spdx_id</code> fields
Explanation trace per recommendation	Confirmed	<code>trace</code> list populated in <code>forward_chain()</code> ; passed to <code>format_trace()</code>
Patent protection warning (not violation)	Confirmed	Correctly implemented in <code>backward_chain()</code> step 3e
Exception isolation per rule	Confirmed	<code>try/except</code> wraps each rule in <code>forward_chain()</code>
eliminated overrides recommended	Confirmed	<code>recommended --=</code> eliminated post-loop
<code>explain_question()</code> covers all facts	Partial	16 of 16 declared facts have entries; additional aliases not covered
Tests/ <code>test_scenarios.py</code>	To verify	Scenarios A, B, C from README should be unit tested in testing branch

9. Testing Branch Checklist

The following items should be validated in the testing branch before merging:

- Rule count: run `len(RULES)` and update README if `!= 57`

- Scenario A (open-source library): facts = {project_type: library, want_weak_copyleft: True, linking_type: dynamic, commercial_use: True} → expected: LGPL-2.1 recommended
- Scenario B (SaaS closed-source): facts = {saas: True, closed_source: True, need_patent_protection: True, commercial_use: True} → expected: Apache-2.0 recommended; GPL-3.0 and AGPL-3.0 eliminated
- Scenario C (public domain): facts = {want_public_domain: True} → expected: Unlicense and CC0-1.0 recommended
- Backward chain — GPL-3.0 + closed_source: expected compatible=False, violation=source_disclosure_required
- Backward chain — MIT + need_patent_protection: expected compatible=True with patent warning
- Edge case: empty facts dict → no violations, warnings only from patent check if triggered
- Edge case: unknown license_id → compatible=False with error explanation

10. Disclaimer

This documentation was produced for a Knowledge Base Systems course project. The system encodes general license properties from public SPDX data and is not a substitute for legal advice. Consult a qualified lawyer for production licensing decisions.